Algorithm:

Step 1: Input & Initialization
a. Read number of vertices V
b. Read adjacency matrix graph[V][V]
c. Create:
path[V] → stores cycle
visited[V] → marks visited vertices

Step 2: Start Setup (findCycle)
a. Initialize:
path[i] = -1 for all i
visited[i] = false for all i

b. Start from vertex 0:
path[0] = 0
visited[0] = true

c. Call solve(1)

Step 3: Recursive Function: solve(pos)

a. Base Case
if pos == V:
if graph[path[V-1]][path[0]] == 1:
return true
else:
return false

b. Try all vertices
for v = 1 to V-1:

Step 4: Safety Check (isSafe(v, pos))
a. Vertex v is valid if:
graph[path[pos-1]][v] == 1
visited[v] == false

Step 5: Place Vertex
a. path[pos] = v
b. visited[v] = true

Step 6: Recurse
a. if solve(pos + 1) == true:
return true

Step 7: Backtrack
a. visited[v] = false
b. path[pos] = -1

Step 8: If All Fail
return false

Step 9: Final Output
a. If solution found → print cycle
b. Else → print "No Hamiltonian Cycle exists"